

VisualEcho

Real-time AI Scene Narration for the Visually Impaired

Student Name	Badr El Malki Berrada
Student Number	21102456
University	Seoul National University of Science and Technology
Course	Computer Vision
Submission Date	June 7, 2026

Abstract

VisualEcho is a voice-controlled scene narration application designed to assist visually impaired users in understanding their physical surroundings. Using a standard webcam, the application captures a frame upon voice command, generates a natural language description using the BLIP-Large vision-language model, and reads the description aloud via Google Text-to-Speech. The entire interaction is hands-free and screen-free, making it fully accessible to users with visual impairments.

Problem & Motivation

Why this project is needed

253 million

people worldwide live
with visual impairment

36 million

people are
completely blind

90%

live in low-income
countries

The Problem

- Visually impaired users cannot interact with screen-based interfaces, menus, or text overlays.
- Existing assistive tools such as screen readers describe UI elements but cannot describe the physical world.
- Real-world navigation and object recognition requires constant assistance from another person.
- There is a significant gap between screen-reader technology and real-world scene understanding.

Proposed Solution

VisualEcho addresses this gap by providing a fully voice-operated application that describes the user's physical environment on demand. The user simply says 'Start' to trigger a scene description, which is then spoken aloud. No screen interaction is required at any point.

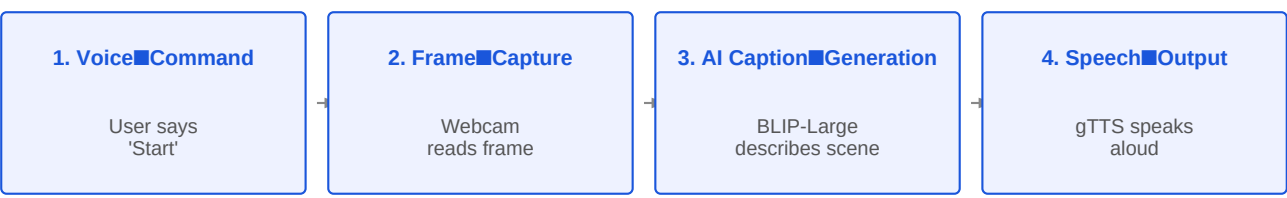
Relevance to Computer Vision

- Real-time image capture and processing using OpenCV
- Vision-language model (BLIP-Large) for scene understanding and caption generation
- Integration of deep learning inference into a practical accessibility application

System Design

Architecture and processing pipeline

Processing Pipeline



Voice Interaction Loop

1. Application starts Speaks welcome message and instructions to the user	2. Listening state Microphone active — waits for 'Start' or 'Quit' command
3. 'Start' detected Acknowledges with 'Analyzing scene, please wait'	4. Frame captured Webcam captures current scene as a still image
5. Caption generated BLIP-Large model produces a natural language description	6. Description spoken gTTS converts caption to audio and plays it aloud
7. Instructions repeated App speaks instructions again and returns to listening state	8. 'Quit' detected App says 'Goodbye' and exits cleanly

Key Design Decisions

- On-demand description (not periodic) — respects user agency and avoids information overload
- Audio-only primary interface — screen output is supplementary for demo purposes only
- Sequential pipeline (no threads) — simpler, more predictable, easier to debug and demonstrate
- Cross-platform stack — Python, OpenCV, gTTS, SpeechRecognition work on Mac, Windows, and Linux

Technology Stack

Libraries, models, and frameworks

BLIP-Large (Salesforce)

[AI Vision Model — Scene Captioning](#)

Bootstrapping Language-Image Pre-training model, large variant. Takes a single image as input and outputs a natural language description of the scene. Downloaded automatically from HuggingFace on

Rationale: State-of-the-art open-source vision-language model. Free, no API key, runs locally.

SpeechRecognition + PyAudio

[Voice Input — Command Detection](#)

SpeechRecognition captures audio from the microphone via PyAudio and submits it to the Google Speech-to-Text API. The application listens for the keywords 'start', 'quit', and common synonyms.

Rationale: Free, widely used, cross-platform. Google STT provides high accuracy for command detection.

gTTS (Google Text-to-Speech)

[Voice Output — Scene Narration](#)

Converts the AI-generated caption to natural-sounding audio using Google's TTS API. Audio is played via pygame. Selected over pyttsx3 (which uses OS voices) due to significantly better voice

Rationale: Natural voice quality, free, cross-platform. Superior to built-in OS TTS engines.

OpenCV

[Camera Interface — Frame Capture and Display](#)

Handles webcam initialization, frame reading, and the live video window. An on-screen overlay displays the current system state (LISTENING / ANALYZING / SPEAKING) and the last generated

Rationale: Industry standard for computer vision applications. Free, cross-platform, well-documented.

Project Schedule

3-week development plan — May 4 to May 24, 2026

Week 1 | May 4 – May 10

Core Pipeline Development

- Set up Python environment, install all dependencies (OpenCV, Transformers, gTTS, SpeechRec...
- Integrate BLIP-Large model via HuggingFace Transformers, verify frame-to-caption output
- Integrate gTTS — connect caption output to spoken audio playback via pygame
- End-to-end test: webcam frame capture → AI caption → spoken description
- Push initial code to GitHub repository, write basic README skeleton

Deliverable: Working prototype: camera captures a frame and describes it aloud

Week 2 | May 11 – May 17

Voice Control Interface

- Integrate SpeechRecognition + PyAudio, test microphone input on development machine
- Implement voice command loop: detect 'Start' and 'Quit' keywords with synonym matching
- Add welcome message and spoken instruction prompts at startup and after each description
- Handle edge cases: listen timeout, unrecognised commands, network errors for STT/TTS
- Full integration test of voice-only interaction flow, commit progress to GitHub

Deliverable: Fully hands-free, voice-operated application requiring no keyboard or screen interaction

Week 3 | May 18 – May 24

Polish, Documentation, and Submission

- Refine OpenCV overlay: status indicator and caption display for demo/development context
- Cross-platform testing on available OS configurations, resolve any compatibility issues
- Record demonstration video or GIF showing a full session (start, describe, quit)
- Write complete README: installation guide, usage instructions, screenshots, and references
- Final review, clean up commits, zip source code, submit to e-Class before deadline

Deliverable: Polished, fully documented, publicly available open-source project submitted on time

Expected Outcome

Final deliverables and evaluation alignment

Functional Features

Fully voice-controlled interface

No keyboard, mouse, or screen interaction required. The application is operable entirely by voice.

On-demand AI scene description

BLIP-Large generates a natural language description of the scene each time the user says 'Start'.

Natural-sounding speech output

Google TTS provides high-quality, human-sounding narration — significantly better than built-in OS voices.

Live camera overlay for observers

OpenCV window displays system status and captions, useful for demonstrations and grading.

Cross-platform compatibility

Runs on macOS, Windows, and Linux with Python 3.9+. All dependencies are free and open-source.

Graceful error handling

Handles network errors, microphone timeouts, and unrecognised commands with clear spoken feedback.

References

Li, J. et al. (2022). BLIP: Bootstrapping Language-Image Pre-training. arXiv:2201.12086.
HuggingFace Transformers. <https://github.com/huggingface/transformers>
OpenCV. <https://opencv.org/>
SpeechRecognition. https://github.com/Uberi/speech_recognition
gTTS (Google Text-to-Speech). <https://github.com/pndurette/gTTS>